

-  Home
-  Library
-  Profile
-  Stories
-  Stats

-  Following
-  Python in Plain En...
-  Generative AI
-  AI Quick Tips
-  Dev Genius
-  Top Python Librari...
-  Realworld AI Use C...
-  Level Up Coding
-  AI Mind
-  The Useful Tech
-  All About ChatGPT
-  More

Level Up Coding

★ Member-only story

I Asked AI to Build Me a Dev Team in 30 Seconds — And It Actually Worked

You know that feeling when you discover a feature in a tool you’ve been using for months, and you think, “Wait, this was here the whole time?”

 ZIRU

Follow

 11 min read · Apr 6, 2026

 108 



Photo by [Helena Lopes](#) on [Unsplash](#)

That happened to me last week with Claude Code.

I've been using Claude Code as my daily coding companion for a while now. I ask it questions, let it write functions, debug my code — the usual stuff. It's fast, it's helpful, and honestly, it's hard to imagine going back to coding without it.

But here's what I didn't realize. Claude Code isn't just a single AI assistant anymore. There's an experimental feature that lets you build an entire team of specialized AI agents — each with different expertise, different responsibilities, and the ability to coordinate with each other.

When I first heard about this, I thought it was one of

those “sounds cool but probably too complicated to actually use” features. Setting up multiple agents with defined roles, communication protocols, and orchestration patterns? That sounds like something you’d need a PhD in distributed systems to configure.

Then I found a project called Harness that changed my mind completely. It takes the whole process and reduces it to a single sentence: “Build a harness for this project.”

GitHub - revfactory/harness: A meta-skill that designs domain-specific...

A meta-skill that designs domain-specific agent teams, defines...

github.com

But before I tell you about Harness, let me explain why AI agent teams matter in the first place — because once you understand the concept, you’ll see why this is probably the biggest shift in how we use AI for coding.

The Problem With One AI Doing Everything

Think about how a real software team works. You don’t have one person doing architecture, frontend, backend, testing, documentation, and code review all by themselves. That would be insane. Instead, you

have specialists. A frontend developer who knows React inside and out. A backend engineer who thinks in databases and APIs. A QA person whose entire job is to find the things everyone else missed.

Each person focuses on what they're best at, and the result is better than any single person could produce alone.

Now think about how most of us use AI coding tools today. We have one AI assistant. We ask it to write code, review code, debug code, write tests, generate documentation — everything. It's like having one employee who does every job in the company.

The AI does its best. And honestly, it's pretty good at most things. But it's not great at everything simultaneously. When you ask one model to write code and also review that same code, it's essentially checking its own homework. It tends to miss the same blind spots. It optimizes for one thing while missing another.

This is the fundamental problem that multi-agent systems try to solve. Instead of one AI doing everything, you split the work across multiple specialized agents, each focused on one area.

What Are AI Agent Teams?

The idea behind agent teams is simple in concept but powerful in practice.

Imagine you could spin up three AI agents instead of one. The first agent is an Analyst — it reads your codebase, understands the architecture, and identifies what needs to be built. The second agent is a Builder — it takes the Analyst's recommendations and writes the actual code. The third agent is a QA specialist — it reviews the Builder's code, runs checks, and catches problems before they reach production.

Each agent has its own set of instructions, its own focus area, and its own “skills” — specific capabilities tailored to its role. They can pass information to each other, coordinate their work, and produce an output that's better than what any single agent could do alone.

This isn't science fiction. Claude Code already supports this through an experimental feature called Agent Teams. When you enable it, you can define agents in `.claude/agents/` and skills in `.claude/skills/`, and they work together on your project.

The catch? Setting it all up manually is painful.

You need to write agent definition files that describe each agent's role, expertise, and behavior. You need to create skill files that give each agent specific capabilities. You need to decide on an orchestration pattern — how should the agents coordinate? Should they work sequentially, in parallel, or with a supervisor directing traffic? And you need to set up error handling so the whole system doesn't fall apart when one agent produces unexpected output.

For most developers, this setup cost is a dealbreaker. You want to ship features, not spend hours configuring an AI team management system.

That's exactly the problem Harness solves.

Enter Harness: The One-Liner That Builds Your AI Team

Harness is an open-source Claude Code plugin created by Minho Hwang. The concept is elegantly simple — it's a “meta-skill” that designs domain-specific agent teams, defines the specialized agents, and generates all the skills they need.

In other words, it's an AI that builds your AI team for you.

Here's what happens when you type "build a harness for this project" in Claude Code.

First, Harness analyzes your domain. It looks at your project structure, understands the type of work involved, and determines what kind of agent team would be most effective.

Second, it designs the team architecture. Based on your domain, it chooses from six different orchestration patterns and decides whether to use full agent teams or simpler sub-agents.

Third, it generates agent definitions. These are markdown files that go into your `.claude/agents/` directory. Each file defines an agent's role, expertise, behavior guidelines, and communication protocols.

Fourth, it creates skills. Each agent gets custom skill files in `.claude/skills/` that give it specific capabilities tailored to its role and your project.

Fifth, it sets up integration and orchestration — how the agents pass data to each other, handle errors, and coordinate their work.

Sixth, it validates everything with trigger verification and dry-run testing.

All six phases happen automatically. The output is a fully configured multi-agent setup, ready to use.

How to Install Harness (Step by Step)

Getting Harness up and running takes about 5 minutes. Here's exactly how to do it, straight from the [official repository](#).

Step 1: Enable Agent Teams in Claude Code

This is the most important prerequisite. Agent Teams is still an experimental feature, so you need to turn it on manually. Add this environment variable to your shell configuration file (`.bashrc` , `.zshrc` , or whatever you use):

bash

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

Then restart your terminal or run `source ~/.zshrc` to apply the change. Without this, Harness won't be able to create agent teams, so don't skip this step.

Step 2: Install Harness via the Plugin Marketplace

(Recommended)

This is the easiest method. Open Claude Code and run these two commands:

```
# First, add the marketplace repository
/plugin marketplace add revfactory/harness

# Then install the plugin
/plugin install harness@harnes
```

That's it. Harness is now installed and ready to use.

Step 3 (Alternative): Direct Installation as a Global Skill

If you prefer not to use the marketplace, you can install Harness manually by cloning the repository and copying the skill files:

```
# Clone the repository
git clone https://github.com/revfactory/harness.git

# Copy the skills directory to your global Claude skills
cp -r harness/skills/harness ~/.claude/skills/harness
```

This makes Harness available as a global skill across

all your Claude Code projects.

Step 4: Verify the Installation

Once installed, you can verify everything is working by opening Claude Code in any project and typing:

```
Build a harness for this project
```

Or any variation like “Design an agent team for this domain” or “Set up a harness.” Harness will kick off the six-phase workflow and start generating your agent team.

After it finishes, check your project directory. You should see new files in `.claude/agents/` (agent definitions) and `.claude/skills/` (skill files). If those directories exist and contain markdown files, you're good to go.

The Six Architecture Patterns

One thing that impressed me about Harness is that it doesn't take a one-size-fits-all approach. It supports six

different architecture patterns, and it selects the right one based on your project's needs.

The Pipeline pattern is for sequential dependent tasks. Think of a data processing workflow where you need to extract data, transform it, then load it. Each step depends on the previous one, so they run in order. This is the simplest pattern and works well when you have a clear, linear workflow.

Fan-out/Fan-in is for parallel independent tasks. If you have five things that need to happen but none of them depend on each other, why do them one at a time? This pattern sends work out to multiple agents simultaneously and merges the results when they're all done. It's great for things like running multiple types of analysis on the same codebase.

Expert Pool is like having a bench of specialists on call. The system evaluates the situation and routes the work to whichever agent has the right expertise. Security issues go to the security agent. Performance problems go to the performance agent. It's context-dependent selection rather than a fixed workflow.

Producer-Reviewer is my personal favorite for code generation. One agent writes code, and a completely different agent reviews it. This solves the "checking

your own homework” problem I mentioned earlier. The reviewer has different priorities and catches things the producer would naturally miss.

Supervisor puts a central agent in charge. This supervisor agent receives tasks, evaluates them, and dynamically distributes work to other agents based on current needs. It’s more flexible than a pipeline but requires a smart coordinator.

Hierarchical Delegation is for really complex projects. A top-level agent breaks down work into sub-tasks and delegates them to sub-agents, who might break them down further and delegate to their own sub-agents. It’s recursive delegation, and it works well for large, multi-layered projects.

The beauty of Harness is that you don’t need to understand these patterns upfront. You describe your project, and Harness picks the right one for you.

Two Execution Modes

Harness gives you two ways to run your agent team, and understanding the difference helps you get better results.

Agent Teams mode is the default and the more powerful option. It uses Claude Code’s TeamCreate,

SendMessage, and TaskCreate system, where agents can actively communicate with each other, pass work back and forth, and collaborate on shared outputs. This is recommended when you have two or more agents that need to work together on interconnected tasks.

Sub-agents mode is simpler. Each agent is invoked directly through the Agent tool, handles its task independently, and returns a result. There's no inter-agent communication. This works well for one-off tasks where you just need a specialist to handle something specific without coordinating with other agents.

The choice between these modes depends on your project. Complex projects with lots of interdependencies benefit from full agent teams. Simpler projects with isolated tasks do fine with sub-agents.

Real-World Use Cases

The [Harness repository](#) comes with example prompts that show the range of what's possible. You can copy any of these directly into Claude Code after installing the plugin.

For deep research, you can set up a team where one

agent searches the web, another digs into academic sources, a third monitors community sentiment, and a supervisor cross-validates their findings before producing a comprehensive report. Each agent brings a different perspective, and the final output is more thorough than what any single search could produce.

For full-stack web development, the team handles design, frontend (React/Next.js), backend (API), and QA testing in a coordinated pipeline from wireframe to deployment. Instead of context-switching between different aspects of the project yourself, each agent stays focused on its specialty.

For code review, multiple agents check architecture, security vulnerabilities, performance bottlenecks, and code style in parallel, then merge all findings into a single unified report. This is the fan-out/fan-in pattern in action, and it catches the kind of cross-cutting issues that a single reviewer would miss.

For technical documentation, agents analyze your endpoints, write descriptions, generate usage examples, and review everything for completeness. Documentation is one of those tasks that developers dread doing manually, and having a specialized team handle it is genuinely useful.

There are also examples for webtoon production, YouTube content planning, marketing campaigns, and data pipeline design. The tool isn't limited to pure software development — it can orchestrate agents for any domain where specialized roles and coordination add value.

Harness 100: A Library of Pre-Built Teams

Beyond the core plugin, there's a companion project called Harness-100 that takes things even further. It's a collection of 100 production-ready agent team configurations across 10 different domains, all available in both English and Korean (200 packages total).

Each harness ships with 4–5 specialist agents, an orchestrator skill, and domain-specific skills. The whole collection spans 1,808 markdown files covering content creation, software development, data/AI, business strategy, education, legal, healthcare, and more.

Every single one of these configurations was generated by the Harness plugin itself. So it's not just a tool that builds teams — it's a tool that has proven it can build high-quality teams across a wide range of domains.

Think of Harness-100 as a library of blueprints. Even if you don't use them directly, they're excellent examples of how to structure agent teams for different types of work.

The Data: Does It Actually Work?

Here's where Harness really stands out from most open-source tools. The creator didn't just build it and hope for the best. He ran a controlled A/B test across 15 software engineering tasks, and published the results in a companion repository called [claude-code-harness](#).

The study covered tasks at three difficulty levels — Basic, Advanced, and Expert. For each task, output quality was measured both with and without Harness pre-configuration.

The results were striking. Average quality scores went from 49.5 without Harness to 79.3 with it — a 60% improvement. The Harness-configured agents won every single one of the 15 tasks, for a 100% win rate. And output variance dropped by 32%, meaning results were not just better but significantly more consistent.

But the most interesting finding was how improvement scaled with difficulty. Basic tasks improved by 23.8 points. Advanced tasks improved by

29.6 points. Expert tasks improved by 36.2 points.

In other words, the harder the task, the more Harness helped. This makes intuitive sense — simple tasks don't require much coordination, but complex tasks benefit enormously from structured roles and clear orchestration. It's the difference between one person trying to juggle everything and a well-organized team where everyone knows their job.

The Honest Limitations

No tool is perfect, and I want to be upfront about the things you should consider before diving in.

Agent Teams is still an experimental feature in Claude Code. “Experimental” means the API could change, features might break with updates, and the behavior may not be fully stable yet. This isn't a production-hardened enterprise tool — it's cutting-edge functionality that's still being refined.

For simple, single-task projects, Harness is overkill. You don't need an AI team to fix a typo or write a utility function. The setup and coordination overhead only pays off when the task is complex enough to benefit from specialization.

And as with any AI tool, the output quality depends on

how well you describe what you need. A vague “build something cool” will produce vague results. A specific “build a harness for a full-stack e-commerce platform with React frontend, Node.js backend, and PostgreSQL database” will produce something much more useful.

Why This Matters Beyond Today

We’re at an interesting inflection point in AI-assisted development. For the past two years, the paradigm has been “one AI assistant, one conversation.” You chat with an AI, get help, and move on. It’s useful, but it’s fundamentally limited by the same constraint that limits a single human — one perspective, one set of priorities, one blind spot pattern.

Multi-agent systems break through that limitation. By splitting work across specialized agents with different roles and different perspectives, you get output that’s more thorough, more consistent, and more reliable than what any single agent can produce.

Harness makes this transition practical for everyday developers. You don’t need to understand orchestration theory. You don’t need to write complex agent definitions from scratch. You just describe your project and let the tool handle the complexity.

The repository is open source under an Apache 2.0

license. Whether you're already using Claude Code or just curious about where AI-assisted development is heading, it's worth checking out.

The future of AI coding isn't one assistant doing everything. It's a team of specialists working together. And that future is already here — it just takes 30 seconds to set up.

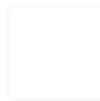
Harness Ai

AI

Coding

Vibe Coding

Software Development



Published in Level Up Coding

Following

317K followers · Last published 2 days ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev



Written by ZIRU

Follow



2.8K followers · 176 following

Lead Principal Data Scientist | Saab Inc. |
Depence AI Specialist | San Diego | UCSD |
<https://www.linkedin.com/in/jhbaek/>

No responses yet



Pancake Agree

What are your thoughts?

More from the list: "Implement"

Curated by Pancake Agree



Mark Chen

**I Used Claude Code to
Build a Personal...**

★ · Apr 6



In Vibe Cod... by Alex Dun...

**The AI Drop That
Changes Everything. Is.**

★ · 4d ago



[View list](#)

More from ZIRU and Level Up Coding



In Level Up Coding by ZIRU

Why 90% of Claude Code Users Are Missin...

The secret isn't better prompts or smarter models — it's one...



Mar 28



213



5



In Level Up Coding by David Lee

The Rules NASA Uses to Write Code That Can't...

In 2006, a NASA engineer wrote ten rules for writing code wher...



Mar 18



3.9K



73



In Level Up Codi... by Fareed Kh...

Building Claude Code with Harness...

Multi-agents, MCP, skills system, context pipelines and...



Apr 6



1.2K



9



ZIRU

How I Set Up OpenClaw on My Mac Mini M4 —...

Everything I learned after weeks of tuning, debugging, and...



Feb 16



339



14



See all from ZIRU

See all from Level Up Coding

Recommended from Medium

 In Vibe Coding by Alex Dunlop

I Cut Claude Code's Output Tokens by 75%....

Fewer tokens. Somehow more accurate. There's a paper.

★ 6d ago

 992

 12



 Kristopher Dunham

MemPalace: The Viral AI Memory System That...

Every AI agent you've ever built has the same fatal flaw.

...★ Apr 8

 604

 8



...

In Towards AI by Rick Hightower

Claude Code 2026: The Daily Operating Syste...

The exact 5-part model, 10-minute routine, slash...

★ Apr 7 🖱 292 💬 ...

In The Ai Studio by Ai studio

What Is Andrej Karpathy's CLAUDE.m...

And Why 10,000+ Developers Downloaded It

★ Apr 7 🖱 218 💬 1 💬 ...

In Data Science ... by Gao Dalie (...)

Gemma 4 + Turboquant + RAG: Better OCR &...

If you don't have a Medium subscription, use this link to...

★ Apr 8 🖱 361 💬 1 💬 ...

In AI Advances by Yanli Liu

I Ran Codex and Claude Side by Side. Here's...

The setup takes 5 minutes. The compliance question takes...

★ 5d ago 🖱 424 💬 3 💬 ...

See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Rules](#) [Terms](#)
[Text to speech](#)